

Notification Framework

A unified multi-channel notification system using FastAPI · RabbitMQ · Celery · MongoDB

FastAPI

RabbitMQ

Celery

MongoDB

React

Problem Statement

Why do we need a centralised notification service?

Fragmented channels

Email, SMS, WhatsApp, Push handled by separate codebases with no coordination.

No retry logic

Failed deliveries are silently dropped with no backoff or dead-letter handling.

No user control

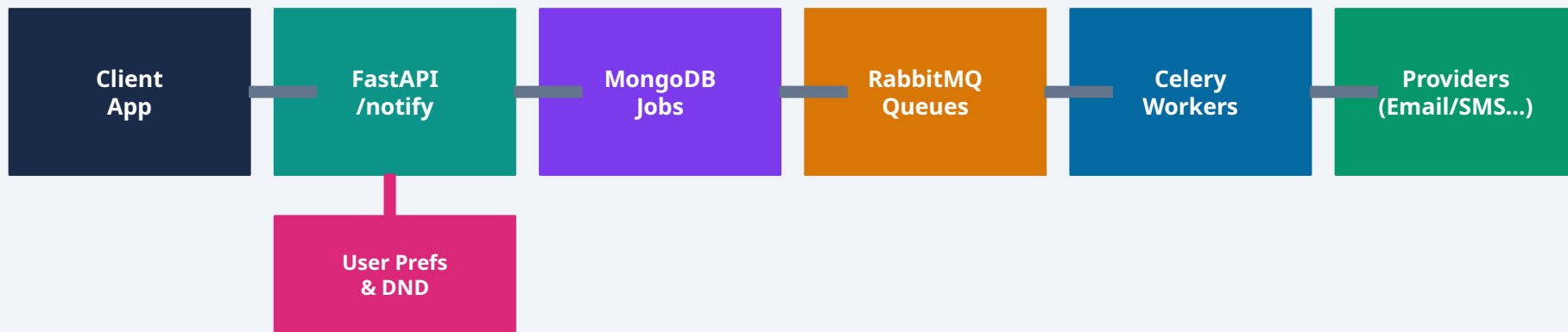
Users cannot set preferences, opt-out of channels, or configure DND windows.

No audit trail

Zero visibility into delivery status — sent, failed, or still queued.

System Architecture

End-to-end flow from event trigger to provider delivery



User preferences and DND windows are checked before any job is queued

FastAPI — /notify Endpoint

Core orchestration layer

- 1 Store raw request** Saves incoming payload to notification_requests collection immediately.
- 2 Resolve channels** DND check + user preference intersection per recipient.
- 3 Create job docs** One MongoDB job per (recipient × channel) with retry_policy.
- 4 Dispatch to Celery** apply_async() routes each job to the correct RabbitMQ queue.
- 5 Return summary** Response includes request_id, jobs_created, and jobs_blocked.

User Preferences & DND Engine

Fine-grained per-user notification control

Preference Resolution Order

- ① No user prefs → honour caller channels
- ② Global DND window active → block ALL
- ③ Per-event DND active → block for event
- ④ Intersect user channels n caller channels

DND Config Example

```
"dnd_windows": [{  
  
  "days": ["Mon", "Tue"],  
  
  "start": "22:00",  
  
  "end":    "07:00",  
  
  "tz":     "Asia/Kolkata"  
}]
```

Celery Workers & RabbitMQ Queues

Asynchronous, per-channel job processing

email-notify-q

Provider: SendGrid

Worker: --concurrency=4 | acks_late=True |
prefetch=1

whatsapp-notify-q

Provider: UltraMsg

Worker: --concurrency=4 | acks_late=True |
prefetch=1

sms-notify-q

Provider: Twilio

Worker: --concurrency=4 | acks_late=True |
prefetch=1

push-notify-q

Provider: Firebase FCM

Worker: --concurrency=4 | acks_late=True |
prefetch=1

MongoDB Data Model

Four collections — audit trail from request to delivery

notification_requests

- request_id
- client_id
- event_type
- recipients[]
- channels_requested
- status
- created_at

notification_jobs

- job_id
- request_id
- channel
- recipient_address
- priority
- retry_policy
- status

delivery_logs

- log_id
- job_id
- provider
- provider_message_id
- status
- attempt_number
- sent_at

user_preferences

- user_id
- email / phone / wa_number
- event_channel_preferences[]
- dnd_windows[]

Frontend — React + Vite

User Preferences UI

UI Capabilities

- Set email, phone, WhatsApp, FCM token
- Choose channels per event type
- Configure global & per-event DND windows
- Timezone-aware DND scheduling
- Calls GET/POST /preferences on FastAPI
- Calls /user-event-types to populate dropdown

Tech Stack

Framework	React 19 + Vite 8
Build	ES Modules (type: module)
Dev server	vite --port default 5173
API proxy	CORS allowed origin *
State	React useState / useEffect

Sprint 2 — What's Next

Planned features for the next sprint

Core

Admin Dashboard

Monitor delivery stats, job statuses, DLQ entries and queue health in real time

Core

Retry with Exponential Backoff

Configurable per-job backoff [10s → 30s → 2m → 10m → 30m] with DLQ on max attempts

Core

Registration Token — API Key Management

Issue & validate API keys per client_id; replace open /notify with authenticated access

Core

Additional Notification Channels

Extend Unified Notification API with new channel types (e.g. In-App, Slack, Teams)

Optional

Channel Fallback Mechanism

If primary channel fails after retries, auto-escalate to a configured fallback channel